

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA0304

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

- 1. Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

(10 Marks)
- 2. Design and implement an Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

(10 Marks)
- 3. Develop a Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

(10 Marks)

4. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.
(10 Marks)
5. **Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.
(10 Marks)
6. **Design and develop a Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.
(10 Marks)
7. **Develop a Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.
(10 Marks)
8. **Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.
(10 Marks)
9. **Develop a Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.
(10 Marks)
10. **Design, implement, and evaluate a Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.
(10 Marks)

Code of Conduct:

I (k varshan / 192325089) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K VARSHAN

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

REINFORCEMENT LEARNING – POLICY-BASED RL ASSESSMENT

1. Policy Gradient-Based RL for Intelligent Traffic Signal Control

Aim

To design and develop a Policy Gradient-based Reinforcement Learning system for controlling traffic signals and reducing vehicle waiting time while improving traffic throughput.

Problem Design

The traffic signal is treated as an RL environment.

State Space:

Number of vehicles waiting in each lane

Current traffic signal

Queue length

Average waiting time

Traffic density

Action Space:

0 → Keep current signal

1 → Switch to North-South

2 → Switch to East-West

Reward Function:

$$[$$

$$R_t = -(\text{Total Waiting Time})$$

$$]$$

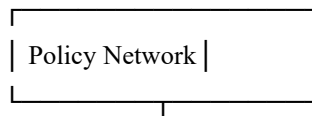
Additional positive reward can be given when the number of vehicles passing through the junction increases.

Architecture

Traffic Environment



State Vector



Probability of Actions



Traffic Signal



New State + Reward



Policy Update

Algorithms

REINFORCE

REINFORCE directly updates the policy using the accumulated return:

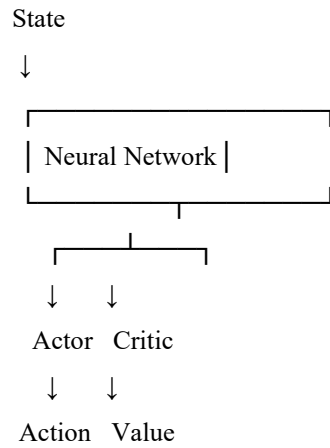
$$[$$

$$\nabla J(\theta) = E[\nabla \log \pi_{\theta}(a|s) G_t]$$

$$]$$

A2C

A2C uses both an actor and a critic.



The advantage is:

$$A(s,a) = Q(s,a) - V(s)$$

Python Implementation

```

import numpy as np
import tensorflow as tf
from tensorflow.keras import layers

class TrafficPolicy(tf.keras.Model):
    def __init__(self, actions):
        super().__init__()
        self.d1 = layers.Dense(64, activation="relu")
        self.d2 = layers.Dense(64, activation="relu")
        self.out = layers.Dense(actions, activation="softmax")

    def call(self, state):
        x = self.d1(state)
        x = self.d2(x)
        return self.out(x)

policy = TrafficPolicy(3)
optimizer = tf.keras.optimizers.Adam(0.001)

def select_action(state):
    probabilities = policy(

```

```

        tf.convert_to_tensor([state], dtype=tf.float32)
    )
    action = tf.random.categorical(
        tf.math.log(probabilities), 1
    )[0, 0]
    return int(action.numpy())

```

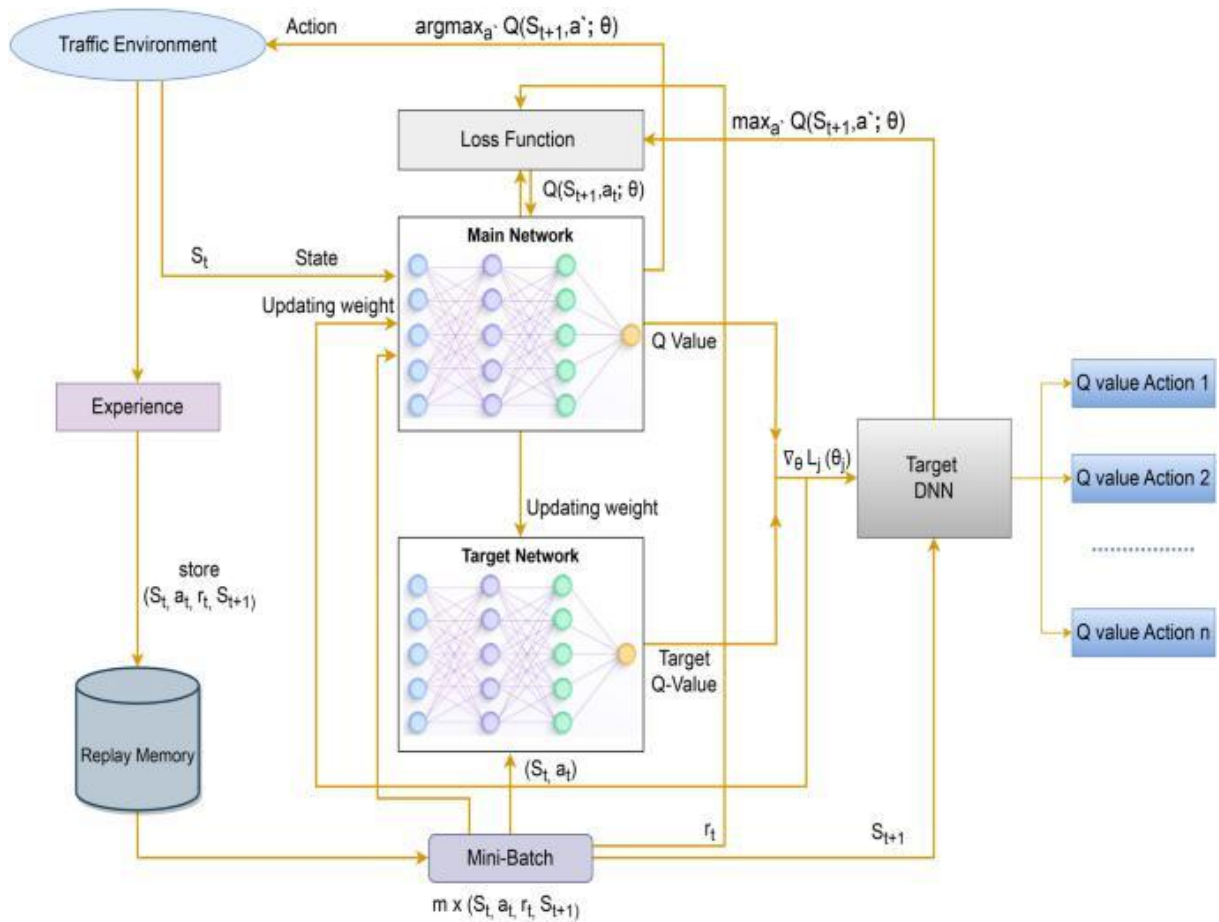
Evaluation Metrics

Metric	REINFORCE A2C	
Average waiting time	Higher	Lower
Throughput	Moderate	Higher
Training stability	Lower	Higher
Convergence	Slower	Faster

Expected Result

A2C is expected to converge faster than REINFORCE because the critic provides a baseline for reducing policy-gradient variance. The intelligent signal controller should reduce queue length and average waiting time.

Diagram



Conclusion

Policy Gradient methods can learn traffic signal policies directly from traffic conditions. A2C generally provides more stable learning than REINFORCE for this problem.

2. Actor-Critic RL for Autonomous Warehouse Robot

Aim

To develop an Actor-Critic model for an autonomous warehouse robot that collects packages and delivers them to their appropriate destinations.

Environment Design

The warehouse is represented as a grid.

Where:

R = Robot

P = Package

D = Delivery point

X = Obstacle

S = Starting point

State Space

[
S=[x,y,p,d,o]
]

where the state contains:

Robot position

Package position

Delivery position

Nearby obstacles

Package status

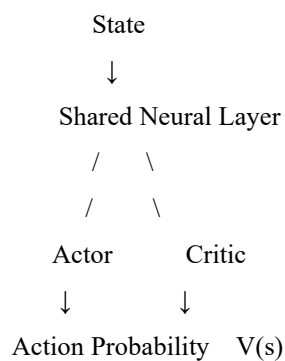
Action Space

0 → Up
1 → Down
2 → Left
3 → Right
4 → Pick-up
5 → Deliver

Reward Function

Reach package → +10
Successful delivery → +50
Move → -1
Collision → -20
Wrong action → -5

Actor-Critic Architecture



The actor chooses the action while the critic estimates the value of the current state.

Python/PyTorch Implementation

```
import torch
import torch.nn as nn

class ActorCritic(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()

        self.shared = nn.Sequential(
```

```

nn.Linear(state_size, 128),
nn.ReLU(),
nn.Linear(128, 64),
nn.ReLU()
)

self.actor = nn.Linear(64, action_size)
self.critic = nn.Linear(64, 1)

def forward(self, state):
    x = self.shared(state)

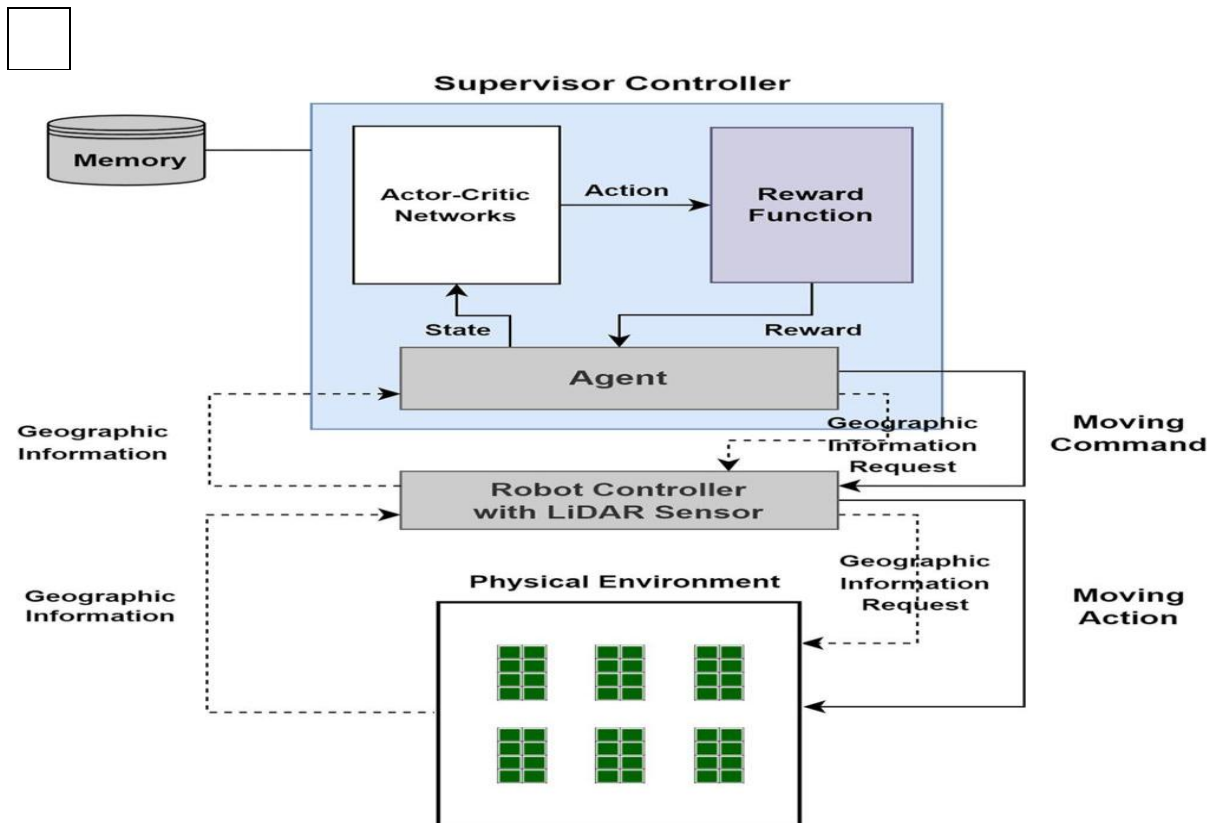
    action_prob = torch.softmax(
        self.actor(x), dim=-1
    )

    value = self.critic(x)

    return action_prob, value

```

Diagram



Comparison

Metric	Vanilla Policy Gradient	Actor-Critic
Reward	Moderate	Higher
Training stability	Lower	Higher
Task completion time	Longer	Shorter
Variance	High	Lower
Convergence	Slow	Faster

Expected Result

Actor-Critic should complete package delivery with fewer unnecessary movements and achieve higher cumulative reward.

Conclusion

Actor-Critic is more suitable for warehouse robotics because the critic provides value estimation that improves training stability and reduces policy-gradient variance.

3. DDPG for Continuous Portfolio Optimization

Aim

To develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization and evaluate its return, risk, and convergence.

Problem Design

The RL agent acts as a portfolio manager.

State Space

The state contains:

Stock prices

Historical returns

Trading volume

Moving averages

Portfolio balance

Previous allocation

Market indicators

For example:

[
S_t=[P_t,R_t,V_t,B_t,W_t]
]

Action Space

DDPG is suitable because portfolio allocation is continuous.

For three assets:

$$A = [w_1, w_2, w_3]$$

where:

$$w_1 + w_2 + w_3 = 1$$

Example:

Stock A $\rightarrow$ 0.40

Stock B $\rightarrow$ 0.35

Stock C $\rightarrow$ 0.25

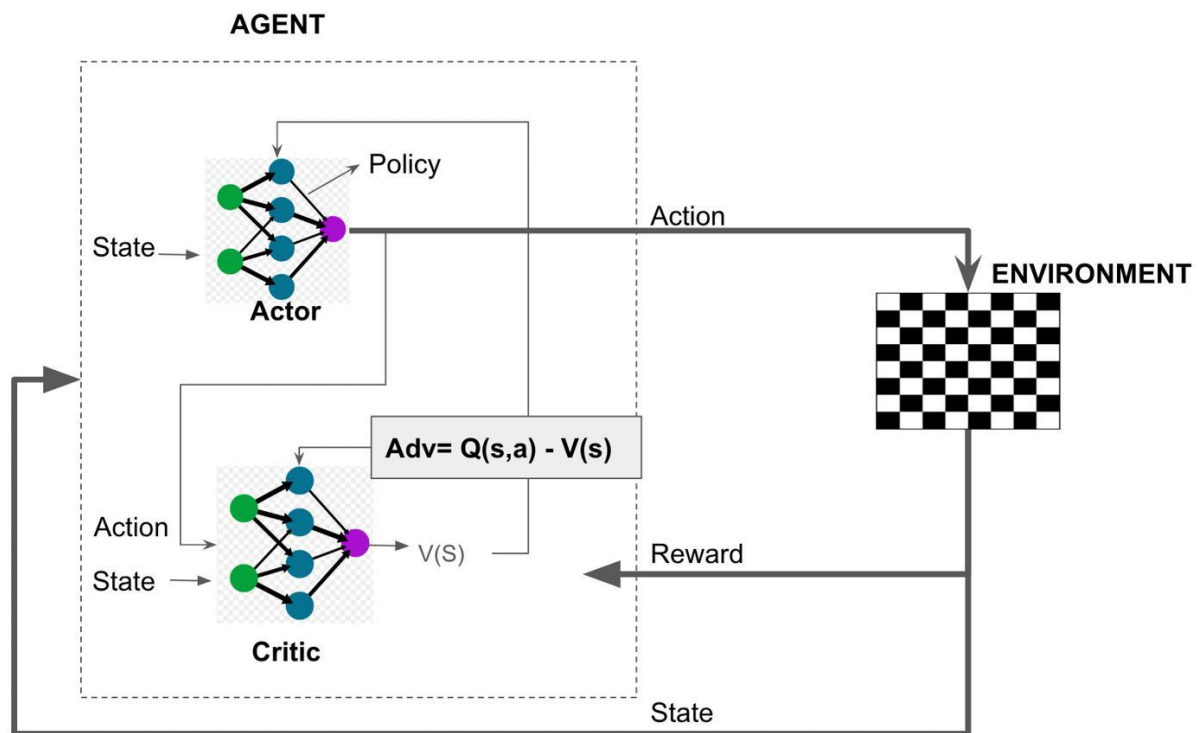
Reward Function

A simple risk-adjusted reward is:

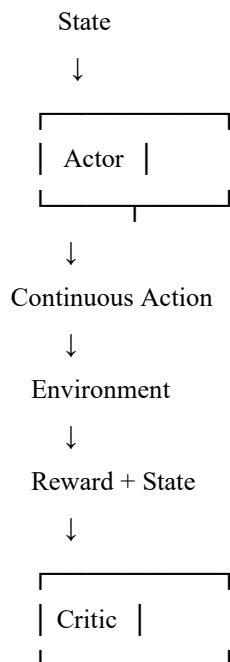
$$R_t = \text{Return}_t - \lambda \text{Risk}_t$$

where λ controls risk sensitivity.

Diagram



DDPG Architecture



DDPG uses:

Actor network

Critic network

Target actor

Target critic

Replay buffer

Evaluation

[
Cumulative\ Return= $\prod_{t=1}^T (1+r_t)-1$
]

Other metrics:

Maximum drawdown

Volatility

Sharpe ratio

Portfolio return

Convergence speed

Expected Result

The DDPG agent should learn continuous portfolio weights rather than selecting only one fixed action. A successful model should achieve a good balance between return and risk.

Conclusion

DDPG is appropriate for portfolio optimization because its continuous action space allows the agent to determine the percentage allocation among multiple assets.

4. PPO for Smart HVAC Energy Management

Aim

To develop a Proximal Policy Optimization (PPO) controller for a smart building HVAC system that minimizes energy consumption while maintaining occupant comfort.

Environment

The building environment provides:

Temperature

Humidity

Occupancy

Outside temperature

HVAC power consumption

Desired temperature

State

[
S_t=[T_{in},T_{out},H,O,P]
]

Action

The action represents HVAC control:

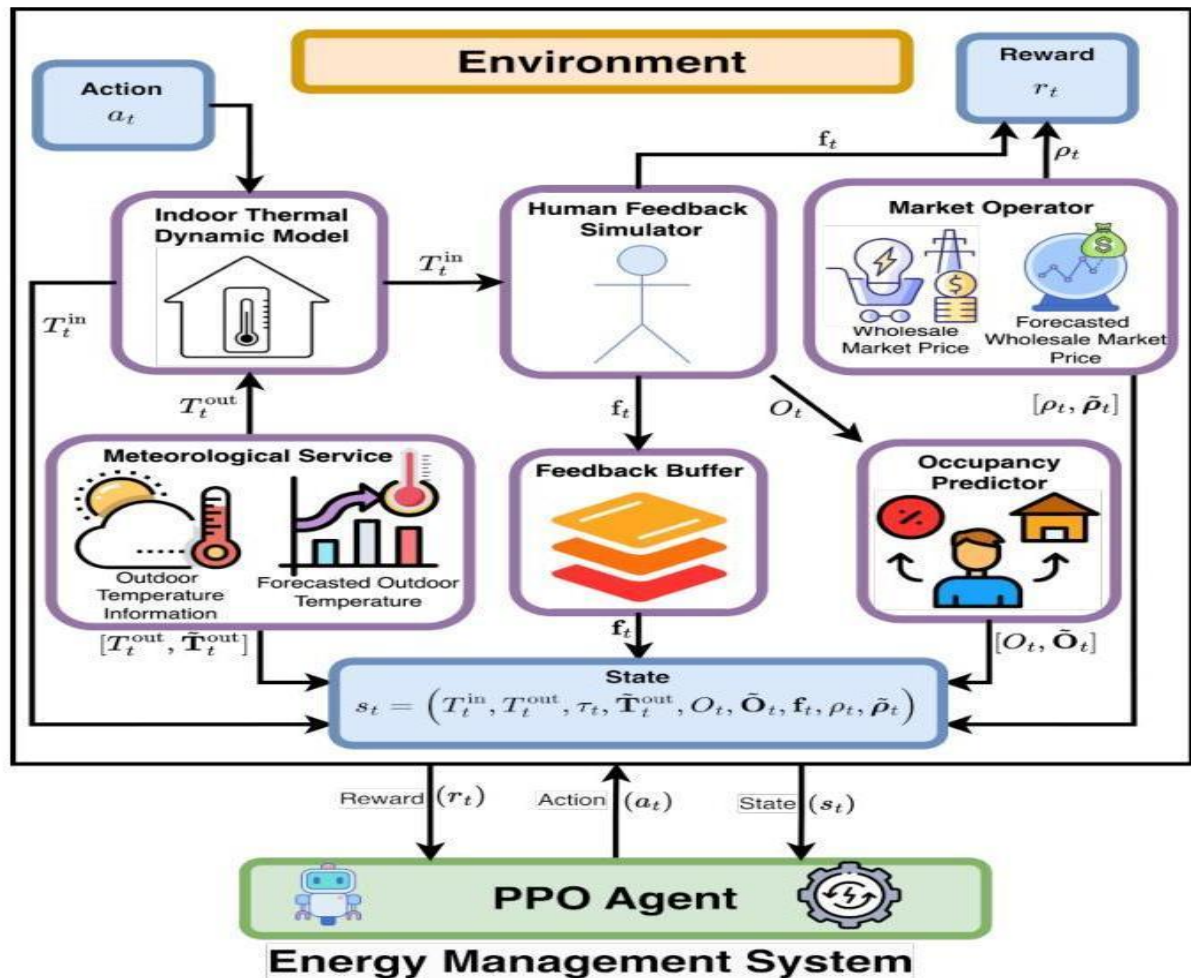
Increase cooling

Decrease cooling

Maintain cooling

Increase heating

Diagram



Decrease heating

A continuous action version can directly control the HVAC temperature set point.

Reward Function

[
 $R_t = -\text{Energy}_t - \lambda |T_t - T_{\text{desired}}|$
]

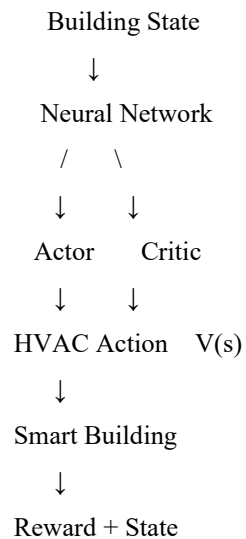
Thus, the agent receives:

Negative reward for excessive energy consumption.

Negative reward for discomfort.

Better reward when comfort is maintained with low energy consumption.

PPO Architecture



PPO uses a clipped objective:

$$L^{\text{CLIP}} = E[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

PPO vs REINFORCE

Metric	REINFORCE	PPO
Energy optimization	Moderate	High
Training stability	Lower	Higher
Policy updates	Large possible changes	Controlled
Convergence	Slower	Faster
Comfort maintenance	Moderate	Better

Expected Result

PPO should reduce HVAC energy consumption while keeping indoor temperature within the acceptable comfort range.

5. TRPO for Industrial Robotic Arm Control

Aim

To implement a Trust Region Policy Optimization algorithm for controlling an industrial robotic arm and improve precision, stability, and production efficiency.

Environment

The robot must move its end-effector from a starting position to a target position.

State Space

Joint angles

Joint velocities

End-effector position

Target position

Motor state

Action Space

Continuous joint movements:

[
A=[a₁,a₂,\ldots,a_n]
]

Reward

[
R=-Distance(target,end\text{-}effector)
]

Additional rewards:

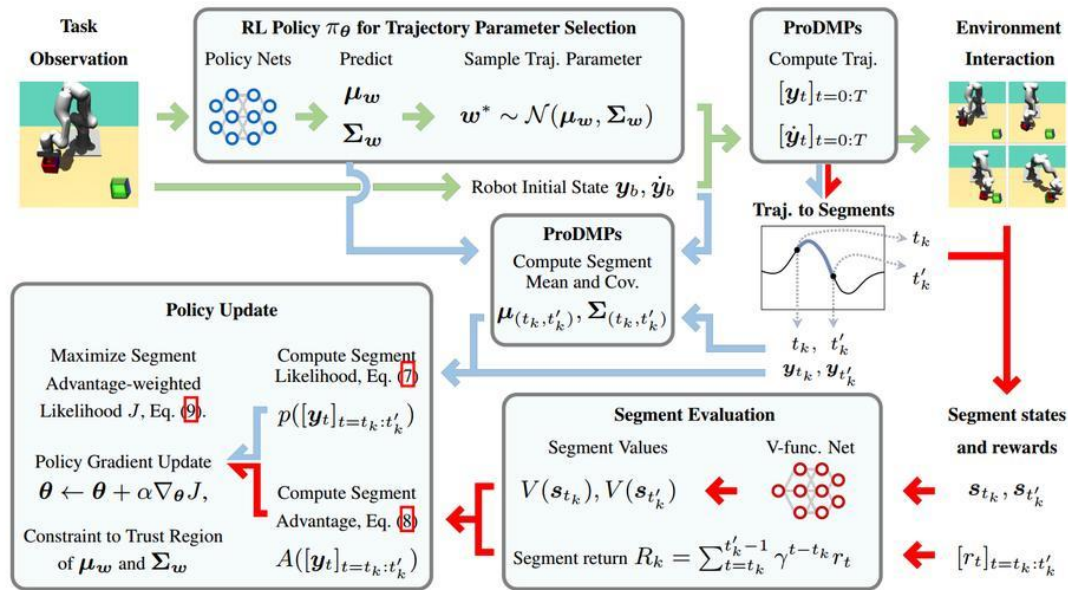
Accurate placement → +100

Successful task → +50

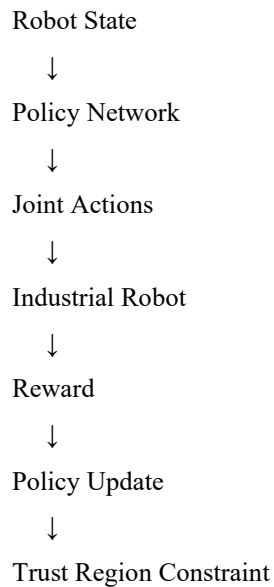
Collision → -100

Large movement $\rightarrow -1$

Diagram



TRPO Architecture



TRPO limits the change between the old and new policies using KL divergence:

$$D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_{\text{new}}) \leq \delta$$

Evaluation Metrics

1. Policy Stability

KL divergence

Reward variance

2. Precision

$$\left[\text{Precision} = 1 - \frac{\text{Position Error}}{\text{Maximum Error}} \right]$$

3. Convergence Speed

Number of training episode

4. Production Efficiency

Tasks completed per unit time

Failure rate

Average cycle time

Expected Result

TRPO should produce stable policy updates and accurate robotic-arm movements. It is particularly useful when large policy changes could result in unsafe robotic behavior.

Conclusion

TRPO's trust-region constraint improves policy stability and makes it suitable for precision-sensitive industrial control tasks.

6. Policy-Based RL for Adaptive Healthcare Treatment Planning

Aim

To design a Policy-Based Reinforcement Learning system that selects adaptive treatment decisions using simulated patient data.

Important Note

The system is intended as a **simulation/research model**, not as a real clinical decision-making system.

Patient State

The simulated patient state can include:

Age group

Disease severity

Previous treatment

Vital-sign indicators

Treatment response

Laboratory measurements

Action Space

Treatment A

Treatment B

Treatment C

Continue current treatment

Modify treatment

Reward

[

$R_t =$

Improvement- λ (AdverseEffects)

]

A successful simulated treatment response receives positive reward, while poor outcomes receive negative reward.

A2C Architecture

Patient State



Neural Network



Actor Critic



Action Value

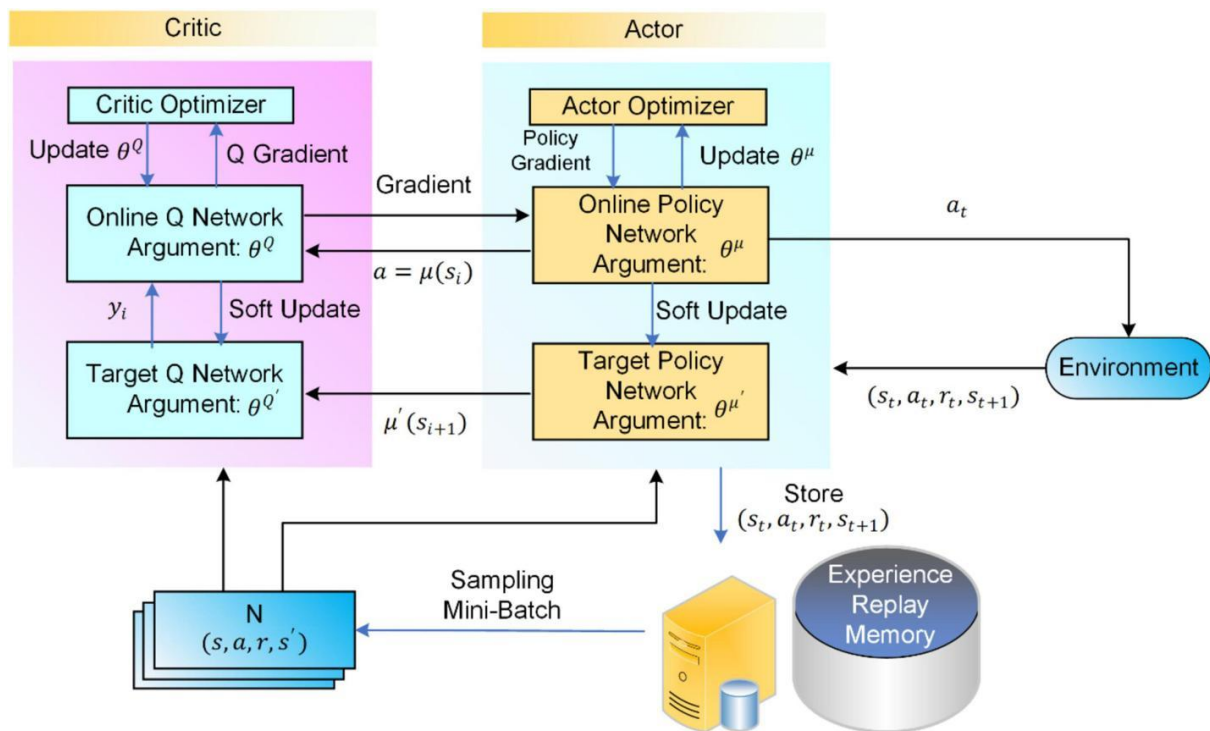
PPO Architecture

PPO uses the actor-critic structure with a clipped policy objective.

Comparison

Metric	A2C	PPO
Treatment reward	High	Higher
Learning stability	Good	Very good
Policy update	Less constrained	Constrained
Convergence	Fast	Stable
Robustness	Good	High

Diagram



Evaluation

The model can be evaluated using:

- Cumulative simulated reward

- Simulated treatment success rate

- Number of adverse simulated outcomes

Expected Result

PPO expected provide more stable learning because restricts excessively large policy updates.

Conclusion

Policy-based RL can be used to investigate adaptive treatment strategies in simulated environments. PPO generally provides more stable optimization than basic A2C.

7. DDPG/PPO for Autonomous Drone Navigation

Aim

To develop a Deep Reinforcement Learning system for autonomous drone navigation in a dynamic environment while avoiding obstacles and minimizing energy consumption.

Environment

The drone operates in a 2D/3D simulated environment.

State Space

Drone position

Drone velocity

Target position

Obstacle distances

Drone orientation

Battery level

Action Space

For PPO/DDPG, actions can control:

Forward velocity

Sideways velocity

Vertical velocity

Rotation

DDPG is particularly suitable for continuous control.

Reward Function

[

R=

$R_{\{goal\}}$

$-R_{\{collision\}}$

$-R_{\{distance\}}$

-R_{energy}
]

Example:

Reach destination $\rightarrow +100$

Avoid obstacle $\rightarrow +5$

Collision $\rightarrow -100$

Excess movement $\rightarrow -1$

Architecture

Sensors

↓

State Representation

↓

RL Policy

↓

Flight Commands

↓

Drone Environment

↓

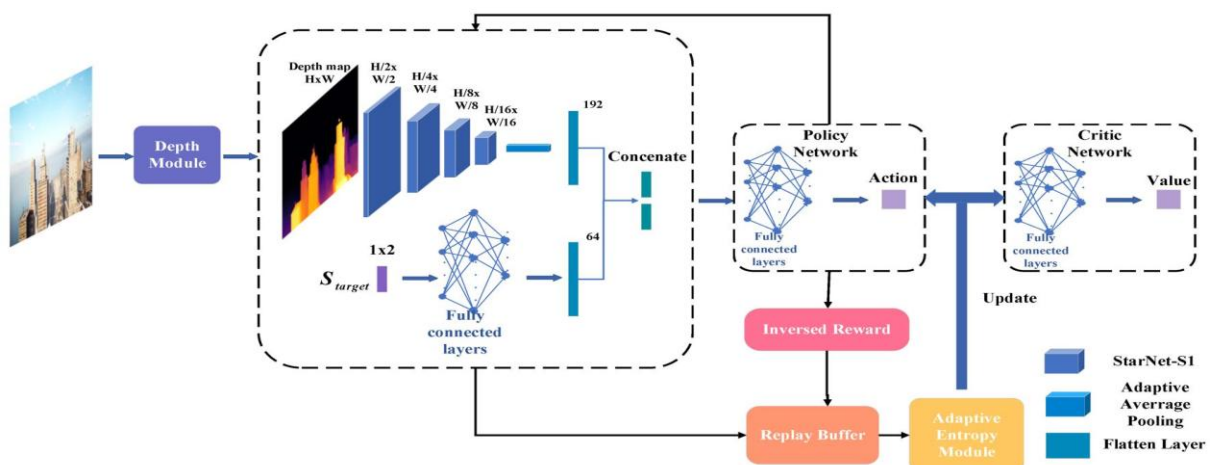
Reward

↓

Policy Update

Evaluation Metrics

Diagram



Navigation Accuracy

$$\text{Accuracy} = 1 - \frac{\text{Final Distance Error}}{\text{Maximum Distance}}$$

Other metrics:

Obstacle collision rate

Successful navigation percentage

Energy consumed

Average path length

Training convergence

Expected Result

The trained drone should learn shorter and safer paths while avoiding dynamic obstacles.

Conclusion

PPO is useful for stable policy optimization, while DDPG is suitable when precise continuous flight-control actions are required.

8. A3C for Dynamic Cloud Resource Allocation

Aim

To design an A3C-based RL framework that dynamically allocates cloud resources according to changing workloads.

Problem Design

The cloud environment contains virtual machines and computing workloads.

State Space

CPU utilization

Memory utilization

Network utilization

Number of active requests

Response time

Available resources

Action Space

Increase CPU

Decrease CPU

Increase memory

Decrease memory

Create VM

Remove VM

Maintain allocation

Diagram

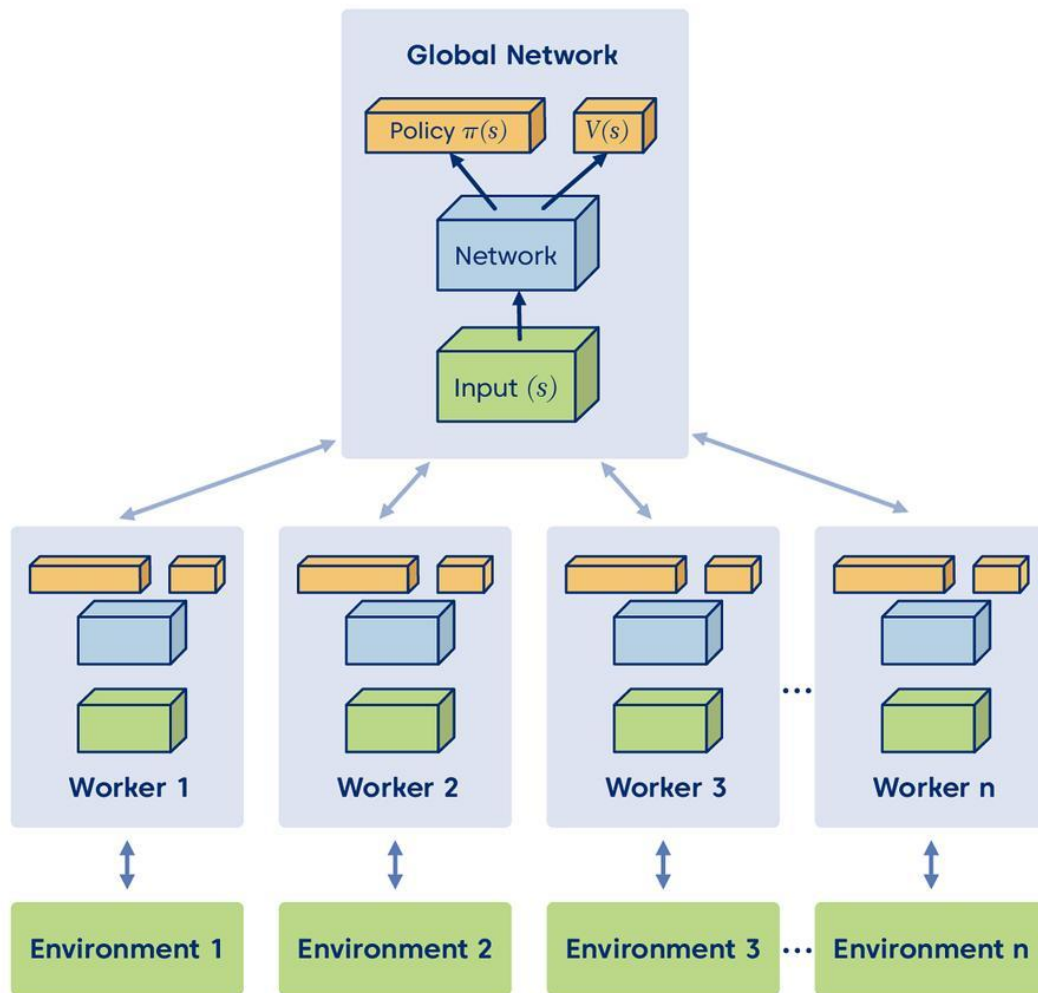


Diagram of A3C high-level architecture.

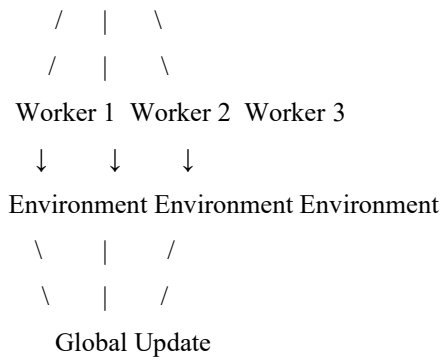
Reward Function

$$\begin{aligned}
 &[\\
 &R = \\
 &-\alpha(\text{Response} \setminus \text{Time}) \\
 &-\beta(\text{Resource} \setminus \text{Cost}) \\
 &+\gamma(\text{Resource} \setminus \text{Efficiency}) \\
 &]
 \end{aligned}$$

A3C Architecture

A3C uses multiple agents/workers operating in parallel.

Global Network



A3C Advantages

Parallel training

Faster experience collection

Reduced correlation between samples

Suitable for dynamic environments

Comparison

Metric	Vanilla Policy Gradient	A3C
Response time	Higher	Lower
CPU utilization	Less optimized	Better
Resource efficiency	Moderate	Higher
Training speed	Slower	Faster
Stability	Moderate	Better

Expected Result

A3C should dynamically adjust resources according to workload, reducing response time and unnecessary resource allocation.

Conclusion

A3C is effective for cloud resource management because multiple workers can learn from different environment states simultaneously.

9. Policy Gradient RL for Predictive Maintenance

Aim

To develop a Policy Gradient-based predictive maintenance system that minimizes machine maintenance cost and downtime.

RL Environment

The machine condition is represented by sensor data.

State Space

Temperature

Vibration

Pressure

Motor current

Operating hours

Previous maintenance

Machine health score

Action Space

0 → Continue operation

1 → Inspect machine

2 → Perform preventive maintenance

3 → Replace component

Reward Function

[

R=

-\text{Maintenance Cost}

-\text{Downtime Cost}

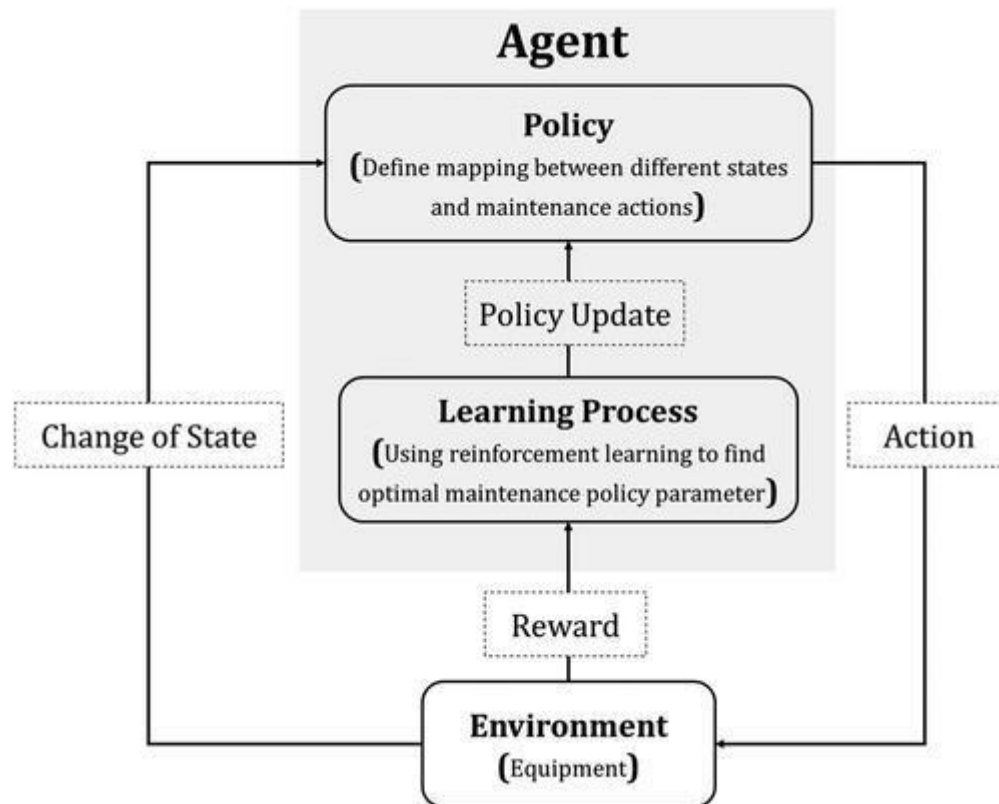
-\text{Failure Penalty}

]

Example:

Normal operation → +5
Preventive maintenance → -10
Machine failure → -100
Successful maintenance → +20

Diagram



Policy Network

Sensor Data
↓
Dense Layer
↓
ReLU
↓
Dense Layer
↓
Softmax
↓
Maintenance Action

REINFORCE

REINFORCE updates the policy using:

$$\left[\nabla_{\theta} J(\theta) \right]$$

$$E[\nabla_{\theta} \log \pi_{\theta}(a|s) G_t]$$

PPO

PPO limits policy updates using a clipped objective and generally provides more stable training.

Comparison

Metric	REINFORCE	PPO
Maintenance cost	Moderate	Lower
Downtime	Higher	Lower
Training stability	Lower	Higher
Convergence	Slower	Faster
Policy robustness	Moderate	High

Expected Result

The agent should learn to perform preventive maintenance before a predicted failure, reducing unexpected breakdowns and downtime.

Conclusion

Policy Gradient RL can transform machine-condition information into maintenance decisions. PPO is expected to outperform REINFORCE in stability and convergence.

10. Policy-Based RL for Autonomous Lane-Keeping

Aim

To design and evaluate a policy-based RL controller for autonomous lane keeping using PPO or TRPO and compare it with A2C.

Environment

The vehicle travels on a simulated road and must remain inside its lane.

State Space

Vehicle speed

Lane center distance

Vehicle heading angle

Lane curvature

Distance from lane boundaries

Steering angle

Action Space

Continuous steering:

[

$A = [-1, 1]$

]

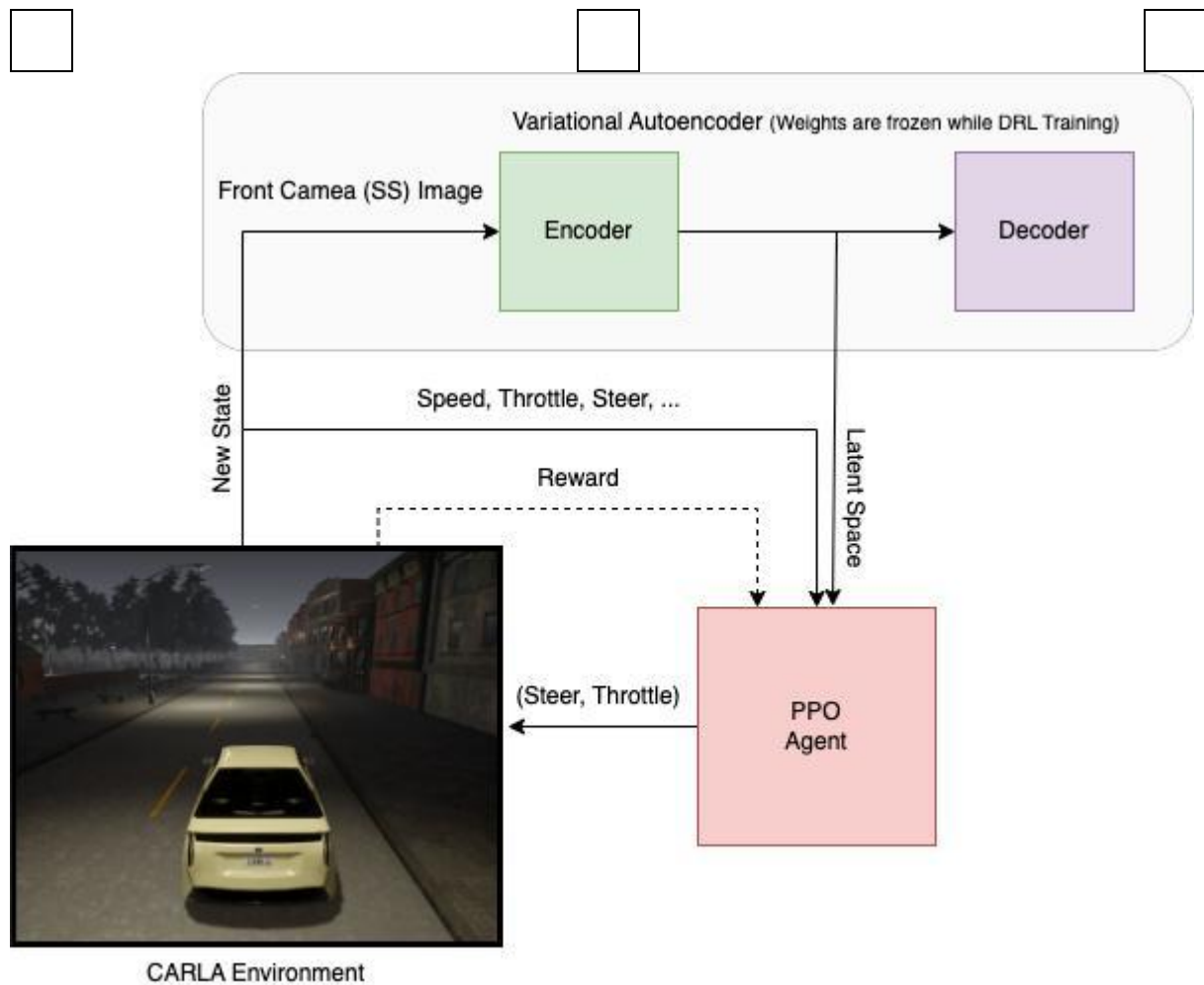
where:

-1 → Maximum left steering

0 → Straight

+1 → Maximum right steering

Diagram



Reward Function

[
 $R =$
 $-\alpha \text{ LaneDeviation}$
 $-\beta \text{ HeadingError}$
 $-\gamma \text{ SteeringChange}$
]

Additional rewards:

Centered in lane $\rightarrow +10$
 Safe driving $\rightarrow +5$
 Lane departure $\rightarrow -100$
 Collision $\rightarrow -200$

PPO Architecture

Camera/Sensor Data



State Processing



Neural Network



Actor



Critic



Steering



V(s)



Vehicle



Lane Position



Reward

PPO vs A2C

Metric	A2C	PPO
Lane deviation	Moderate	Lower
Safety	Good	Better
Reward convergence	Fast	Stable
Training efficiency	Good	High
Policy stability	Good	Very good

Evaluation

Lane Deviation

[

$$LD=\frac{1}{T}\sum_{t=1}^T|d_t|$$

]

where (d_t) is the distance from the lane center.

Other metrics:

Collision rate

Lane departure rate

Average reward

Training episodes

Convergence speed

Steering smoothness

Expected Result

PPO should learn a smoother and more stable lane-keeping policy. The vehicle should maintain a smaller average lane deviation and fewer lane departures.

Conclusion

PPO is well suited for autonomous lane keeping because its controlled policy updates improve training stability. A2C can converge quickly but may require more careful tuning for stable continuous-control behavior.

Overall Experimental Comparison

For the assessment, the following structure can be used when presenting the **results section** of each experiment:

Experiment	Primary Algorithm	Main Application	Main Evaluation
1	REINFORCE + A2C	Traffic signals	Waiting time, throughput
2	A2C/A3C	Warehouse robot	Reward, completion time
3	DDPG	Portfolio optimization	Return, risk
4	PPO	Smart HVAC	Energy, comfort
5	TRPO	Robotic arm	Precision, stability
6	A2C + PPO	Treatment planning	Reward, effectiveness
7	DDPG/PPO	Drone navigation	Accuracy, energy
8	A3C	Cloud resources	Response time, utilization
9	REINFORCE + PPO	Predictive maintenance	Cost, downtime
10	PPO/TRPO + A2C	Lane keeping	Deviation, safety